

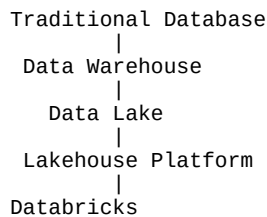
Databricks Master Interview Handbook (2026)

Chapter 1 – Introduction to Databricks & Lakehouse Platform

What is Databricks?

Databricks is a unified analytics platform built on Apache Spark. It enables data engineering, data science, machine learning, streaming, SQL analytics, and AI workloads on a single Lakehouse platform.

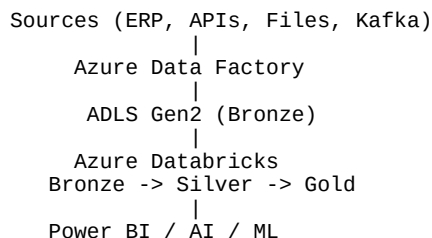
Evolution of Modern Data Platforms



Data Warehouse vs Data Lake vs Lakehouse

A data warehouse is optimized for structured analytics. A data lake stores structured and unstructured data. A Lakehouse combines the reliability of a warehouse with the flexibility and scalability of a data lake.

High-Level Databricks Architecture



Apache Spark Fundamentals

Spark processes distributed data using a driver program and multiple executors. It supports batch processing, streaming, SQL, machine learning, and graph processing.

Databricks Workspace Components

Key components include Workspaces, Notebooks, Clusters, Jobs, Repos, Unity Catalog, SQL Warehouses, Dashboards, and Workflows.

Cluster Types

All-Purpose Clusters are used for interactive development. Job Clusters are created for scheduled jobs and terminate automatically after execution, reducing cost.

Photon & Delta Lake

Photon accelerates SQL and Delta workloads using a vectorized execution engine. Delta Lake provides ACID transactions, schema enforcement, time travel, and scalable metadata.

Interview Questions

1. What is Databricks? 2. Explain the Lakehouse architecture. 3. Difference between Data Lake and Lakehouse. 4. What are Job Clusters and All-Purpose Clusters? 5. What is Photon? 6. Why is Delta Lake important?

Quick Revision

Remember: Databricks = Spark + Delta Lake + Lakehouse + Governance + AI. Explain every concept with a real project example during interviews.

Databricks Master Interview Handbook (2026)

Chapter 2 – Delta Lake Internals (Part 2)

ACID Transactions

Delta Lake guarantees Atomicity, Consistency, Isolation and Durability. Concurrent users can safely read and write data without corrupting the table.

Time Travel

Time Travel lets you query historical versions of a Delta table for auditing, debugging and rollback.

PySpark - Time Travel

```
df = spark.read.format("delta") \
    .option("versionAsOf", 3) \
    .load("/mnt/silver/customers")

df.show()
```

MERGE INTO (UPSERT)

MERGE combines INSERT, UPDATE and DELETE operations in one statement. It is widely used for incremental loading and SCD Type 1/2.

PySpark MERGE Example

```
from delta.tables import DeltaTable

delta_tbl = DeltaTable.forPath(spark, "/mnt/silver/customers")

(delta_tbl.alias("t")
 .merge(source_df.alias("s"), "t.id = s.id")
 .whenMatchedUpdateAll()
 .whenNotMatchedInsertAll()
 .execute())
```

Schema Enforcement vs Schema Evolution

Schema Enforcement rejects incompatible data. Schema Evolution allows new columns to be added when explicitly enabled.

OPTIMIZE and Z-ORDER

OPTIMIZE compacts many small files into fewer larger files. Z-ORDER colocates related data to improve query performance on frequently filtered columns.

OPTIMIZE Example

```
OPTIMIZE sales
ZORDER BY (customer_id);
```

VACUUM

VACUUM removes obsolete files that are no longer referenced by the Delta transaction log, helping reclaim storage.

VACUUM Example

```
VACUUM sales RETAIN 168 HOURS;
```

Production Best Practices

Use MERGE for incremental loads, schedule OPTIMIZE for large tables, run VACUUM carefully after retention requirements, partition wisely, and enable Change Data Feed where downstream incremental processing is needed.

Interview Questions

1. What is Time Travel?
2. Explain MERGE INTO.
3. Difference between Schema Enforcement and Schema Evolution.
4. Why do we use OPTIMIZE and Z-ORDER?
5. What does VACUUM do?
6. How would you implement an incremental load using Delta Lake?

Databricks Master Interview Handbook (2026)

Chapter 2 – Delta Lake Internals (Part 3)

Change Data Feed (CDF)

Change Data Feed tracks row-level INSERT, UPDATE and DELETE operations in Delta tables. It allows downstream applications to process only changed records instead of reading the entire table.

Enable CDF

```
CREATE TABLE sales (  
  id INT,  
  amount DOUBLE  
)  
USING DELTA  
TBLPROPERTIES (  
  delta.enableChangeDataFeed = true  
);
```

Read CDF using PySpark

```
cdf = (spark.read  
  .format("delta")  
  .option("readChangeFeed", "true")  
  .option("startingVersion", 5)  
  .table("sales"))  
  
cdf.show()
```

Delta Table Versioning

Every successful transaction creates a new version of the Delta table. Versions support auditing, rollback and incremental processing.

View History

```
DESCRIBE HISTORY sales;
```

Checkpoint Files

Delta periodically creates checkpoint files to avoid replaying every JSON transaction log. This speeds up table loading and metadata processing.

Optimistic Concurrency Control

Delta uses optimistic concurrency. Multiple writers can work simultaneously, but conflicting commits are detected before completion to preserve consistency.

End-to-End Incremental Load

Typical production flow:

1. Ingest source data.
2. Read only changed records using CDF.
3. Apply MERGE INTO.
4. Validate counts.
5. Publish to Gold layer.

MERGE with Incremental Data

```
deltaTable.alias("t") \  
.merge(cdf.alias("s"), "t.id = s.id") \  
.whenMatchedUpdateAll() \  
.whenNotMatchedInsertAll() \  
.execute()
```

Performance Tips

Enable CDF only where required, partition large tables carefully, schedule OPTIMIZE, use ZORDER on frequently filtered columns, and monitor file sizes to avoid the small-file problem.

Interview Questions

1. What is Change Data Feed?
2. How do you enable CDF?
3. Explain Delta table versioning.
4. What are checkpoint files?
5. Explain optimistic concurrency control.
6. How would you build an incremental pipeline using CDF and MERGE?

Databricks Master Interview Handbook (2026)

Chapter 3 – Unity Catalog (Part 1)

What is Unity Catalog?

Unity Catalog is Databricks' centralized governance solution. It provides a single place to manage data, AI assets, permissions, auditing and lineage across all workspaces.

Why Unity Catalog?

It eliminates inconsistent access control by providing centralized security, fine-grained permissions and unified governance.

Architecture



Object Hierarchy

Metastore is the top-level container. Each Metastore contains Catalogs. Catalogs contain Schemas, and Schemas contain Tables, Views, Volumes and Functions.

SQL Examples

```
CREATE CATALOG sales;

CREATE SCHEMA sales.bronze;

CREATE TABLE sales.bronze.customers(
  id INT,
  name STRING
);
```

Privileges

Common privileges include USE CATALOG, USE SCHEMA, SELECT, MODIFY, CREATE TABLE, CREATE VIEW and ALL PRIVILEGES.

Grant Permission

```
GRANT SELECT
ON TABLE sales.bronze.customers
TO `data_analyst`;
```

Benefits

Centralized governance, row and column security, data lineage, auditing, secure sharing and support for multiple workspaces.

Interview Questions

1. What is Unity Catalog?
2. Explain Metastore, Catalog and Schema.
3. Why is Unity Catalog preferred over Hive Metastore?
4. How do you grant table access?
5. What governance features does Unity Catalog provide?

Databricks Master Interview Handbook (2026)

Chapter 3 – Unity Catalog (Part 2)

Row-Level Security (RLS)

Row-Level Security restricts users to specific rows based on their role or identity. It is commonly implemented using Dynamic Views in Unity Catalog.

Dynamic View Example

```
CREATE VIEW sales.secure_orders AS
SELECT *
FROM sales.orders
WHERE region = current_user();
```

Column-Level Security (CLS)

Column-Level Security protects sensitive columns such as salary, PAN, Aadhaar or customer PII. Only authorized users can view the original values.

Masking Example

```
SELECT
  customer_id,
  CASE
    WHEN is_account_group_member('HR')
    THEN salary
    ELSE NULL
  END AS salary
FROM employees;
```

Data Lineage

Unity Catalog automatically tracks where data originated, which jobs transformed it, and which downstream tables or dashboards consume it.

Storage Credentials & External Locations

Storage Credentials securely store cloud authentication. External Locations map cloud storage paths such as ADLS Gen2, Amazon S3 or Google Cloud Storage for governed access.

Managed vs External Tables

Managed tables store both metadata and data under Unity Catalog management. External tables keep data in customer-managed storage while Unity Catalog manages metadata and permissions.

Unity Catalog Volumes

Volumes allow governance of non-tabular files such as CSVs, images, PDFs, ML models and configuration files.

Interview Questions

1. What is Row-Level Security?
2. How is Column-Level Security implemented?

3. Explain Dynamic Views.
4. What is Data Lineage?
5. Difference between Managed and External Tables?
6. What are Storage Credentials and External Locations?
7. What are Unity Catalog Volumes?

Databricks Master Interview Handbook (2026)

Chapter 3 – Unity Catalog (Part 3)

Delta Sharing

Delta Sharing is an open protocol that allows organizations to securely share live data with external partners without copying data. Consumers always access the latest version.

Create a Share (SQL)

```
CREATE SHARE finance_share;  
ALTER SHARE finance_share  
ADD TABLE finance.gold_transactions;
```

Cross-Workspace Governance

A single Unity Catalog Metastore can govern multiple Databricks workspaces. This provides consistent security policies, permissions and data discovery across development, test and production environments.

Audit Logs

Unity Catalog records operations such as logins, permission changes, table creation and data access. Audit logs support compliance, security investigations and regulatory reporting.

Enterprise Governance Architecture

Typical enterprise setup:

Azure AD -> Unity Catalog -> Metastore -> Catalogs (Bronze, Silver, Gold) -> Schemas -> Tables/Views -> Power BI, ML and external consumers.

All access is controlled using groups and least-privilege permissions.

Best Practices

- Use groups instead of assigning permissions to users.
- Separate Dev, Test and Prod catalogs.
- Enable audit logging.
- Use managed identities/service principals.
- Follow least-privilege access.
- Review permissions periodically.

Real-World Scenario

A global retail company stores Sales, HR and Finance data in separate catalogs. HR users can only access HR data, Finance users only Finance data, while executives receive governed read-only access to Gold reporting tables.

Advanced Interview Questions

1. What is Delta Sharing?
2. How does Unity Catalog support multiple workspaces?
3. Explain audit logs in Unity Catalog.
4. How would you secure PII data?
5. Describe an enterprise governance architecture using Unity Catalog.
6. What are the best practices for managing permissions?

Databricks Master Interview Handbook (2026)

Chapter 4 – Lakeflow Pipelines (Part 1)

What is Lakeflow?

Lakeflow is Databricks' modern orchestration framework for building reliable batch and streaming data pipelines. It simplifies ingestion, transformation, data quality and operational monitoring.

Key Components

- Declarative Pipelines
- Auto Loader
- Auto CDC
- Expectations (Data Quality)
- Pipeline Scheduling
- Event Logs

High-Level Architecture

```
Sources
(SQL, APIs, Files, Kafka)
  |
Auto Loader
  |
Bronze Tables
  |
Transformations
  |
Silver Tables
  |
Aggregations
  |
Gold Tables
  |
BI / AI / ML
```

Batch vs Streaming

Batch pipelines process data at scheduled intervals, whereas streaming pipelines process data continuously as new events arrive.

Data Quality Expectations

Lakeflow supports expectations to validate records, drop invalid rows, quarantine bad data or fail a pipeline when quality rules are violated.

PySpark Example

```
from pyspark.sql import functions as F

df = (spark.readStream
      .format("cloudFiles")
      .option("cloudFiles.format", "csv")
      .load("/mnt/raw/orders"))

clean_df = df.filter(F.col("order_id").isNotNull())

(clean_df.writeStream
 .option("checkpointLocation", "/mnt/chk/orders")
 .toTable("bronze.orders"))
```

Real-World Scenario

An e-commerce company continuously ingests order files into Bronze using Auto Loader, validates mandatory columns, enriches customer data in Silver and publishes daily sales KPIs to Gold.

Interview Questions

1. What is Lakeflow?
2. How is Lakeflow different from traditional ETL?
3. Explain Batch vs Streaming pipelines.
4. What are Expectations?
5. How does Auto Loader fit into Lakeflow?

Databricks Master Interview Handbook (2026)

Chapter 4 – Lakeflow Pipelines (Part 2)

Auto CDC

Auto CDC automatically detects inserts, updates and deletes from supported sources and applies incremental changes to Delta tables, reducing custom ETL code.

Declarative Pipelines

Lakeflow Declarative Pipelines allow engineers to define data transformations and dependencies while Databricks manages execution, optimization and recovery.

Sample Pipeline Flow

```
Source Database
|
Auto CDC
|
Bronze Delta
|
Data Quality Rules
|
Silver Delta
|
Business Aggregations
|
Gold Reporting
```

PySpark Example - Incremental Processing

```
from pyspark.sql.functions import col

bronze_df = spark.read.table("bronze.orders")

silver_df = bronze_df.filter(col("order_status") != "CANCELLED")

(silver_df.write
 .mode("overwrite")
 .saveAsTable("silver.orders"))
```

Pipeline Monitoring

Lakeflow provides pipeline execution history, event logs, data quality metrics and failure details to simplify monitoring and troubleshooting.

Error Handling

Configure expectations to drop invalid rows, quarantine bad records or fail the pipeline. Implement retries and checkpointing for streaming workloads.

Performance Best Practices

Use Auto Loader for file ingestion, partition large tables carefully, enable Photon, use OPTIMIZE and ZORDER, and monitor small-file generation.

Production Scenario

A financial services company ingests transactions every few minutes using Auto CDC. Data quality rules validate mandatory fields, Silver applies business logic, and Gold serves Power BI dashboards with near real-time data.

Interview Questions

1. What is Auto CDC?
2. What are Declarative Pipelines?
3. How do you monitor Lakeflow pipelines?
4. How do you handle bad records?
5. How would you optimize a production Lakeflow pipeline?

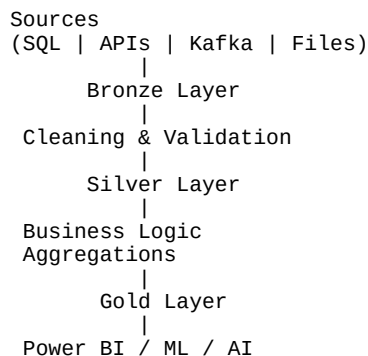
Databricks Master Interview Handbook (2026)

Chapter 5 – Medallion Architecture (Part 1)

What is Medallion Architecture?

Medallion Architecture is a layered data design pattern that organizes data into Bronze, Silver and Gold layers. It improves data quality, governance, scalability and maintainability.

Architecture



Bronze Layer

The Bronze layer stores raw ingested data exactly as received. Minimal transformations are applied, preserving the original data for replay and auditing.

Silver Layer

The Silver layer cleans, validates, deduplicates and enriches Bronze data. Invalid records are handled and business-ready datasets are created.

Gold Layer

The Gold layer contains curated, aggregated and business-focused datasets optimized for reporting, dashboards, machine learning and downstream applications.

PySpark Example - Bronze to Silver

```
from pyspark.sql.functions import col

bronze = spark.table("bronze.orders")

silver = (bronze
          .filter(col("order_id").isNotNull())
          .dropDuplicates(["order_id"]))

silver.write.mode("overwrite").saveAsTable("silver.orders")
```

Real-World Scenario

An online retailer ingests raw order data into Bronze, validates customer and product information in Silver, and publishes daily sales summaries and KPIs in Gold for Power BI.

Interview Questions

1. What is Medallion Architecture?
2. Explain Bronze, Silver and Gold layers.
3. What transformations belong in each layer?
4. Why is Medallion Architecture widely used?
5. How would you implement Bronze to Silver using PySpark?

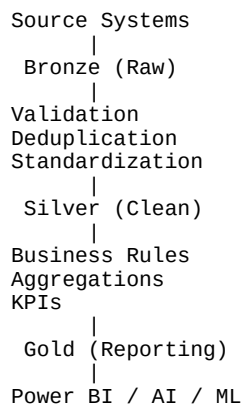
Databricks Master Interview Handbook (2026)

Chapter 5 – Medallion Architecture (Part 2)

End-to-End Medallion Pipeline

Data is ingested into Bronze, cleansed and standardized in Silver, and transformed into business-ready Gold tables. Each layer has a clear responsibility, making pipelines easier to maintain and troubleshoot.

Pipeline Flow



Incremental Processing

Incremental loads use watermark columns, Change Data Feed (CDF), or MERGE operations so only changed records are processed instead of reloading the full dataset.

PySpark MERGE Example

```
from delta.tables import DeltaTable

target = DeltaTable.forName(spark, "silver.orders")

(target.alias("t")
 .merge(source_df.alias("s"),
        "t.order_id = s.order_id")
 .whenMatchedUpdateAll()
 .whenNotMatchedInsertAll()
 .execute())
```

SCD in Medallion

SCD Type 1 overwrites existing values without history. SCD Type 2 preserves historical records by expiring old rows and inserting new versions.

Data Quality

Validate mandatory fields, remove duplicates, quarantine bad records, enforce schema rules and maintain audit columns such as created_date and updated_date.

Performance Optimization

Partition large Delta tables, enable Photon, use OPTIMIZE and ZORDER, cache reused DataFrames, and avoid unnecessary shuffle operations.

Real-World Scenario

A healthcare platform ingests patient events into Bronze, validates records in Silver, applies SCD Type 2 for patient dimensions, and publishes Gold datasets for operational dashboards and analytics.

Interview Questions

1. How do you implement an end-to-end Medallion Architecture?
2. How do you handle incremental loading?
3. Where should SCD Type 2 be implemented?
4. How do you manage bad records?
5. How would you optimize Medallion pipelines in production?

Databricks Master Interview Handbook (2026)

Chapter 6 – Auto Loader (Part 1)

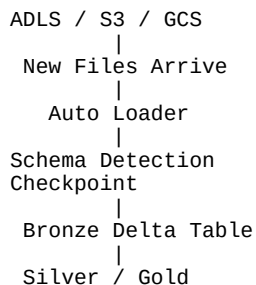
What is Auto Loader?

Auto Loader is Databricks' scalable file ingestion feature built on Structured Streaming. It automatically discovers new files in cloud storage and ingests them incrementally into Delta tables.

Why Auto Loader?

It efficiently processes millions of files, supports schema inference and evolution, avoids duplicate ingestion, and scales better than repeatedly listing cloud storage folders.

Architecture



Discovery Modes

Directory Listing periodically scans folders for new files. File Notification mode integrates with cloud event services for faster and more efficient detection in production.

PySpark Example

```
df = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "csv")
    .option("cloudFiles.schemaLocation", "/mnt/schema/orders")
    .load("/mnt/raw/orders"))

(df.writeStream
    .option("checkpointLocation", "/mnt/checkpoints/orders")
    .toTable("bronze.orders"))
```

Schema Evolution

Auto Loader can automatically detect new columns and evolve the target schema when configured, reducing manual maintenance for changing source files.

Checkpointing

Checkpoint locations store streaming progress and metadata so pipelines can recover safely after failures without reprocessing already ingested files.

Interview Questions

1. What is Auto Loader?
2. Directory Listing vs File Notification?
3. What is schemaLocation?
4. Why are checkpoints required?
5. How does Auto Loader prevent duplicate processing?

Databricks Master Interview Handbook (2026)

Chapter 6 – Auto Loader (Part 2)

Auto Loader Internals

Auto Loader tracks processed files using metadata and checkpoints instead of repeatedly scanning every file. This makes ingestion scalable even when millions of files are present.

Rescued Data Column

When incoming records contain unexpected columns or malformed values, Auto Loader can store them in the `_rescued_data` column instead of failing the entire pipeline. This helps preserve data for later analysis.

Schema Hints

Schema hints allow engineers to explicitly define the data type of selected columns while still allowing Auto Loader to infer the remaining schema.

PySpark Example – Rescued Data

```
df = (spark.readStream
      .format("cloudFiles")
      .option("cloudFiles.format", "json")
      .option("rescuedDataColumn", "_rescued_data")
      .load("/mnt/raw/events"))

df.writeStream.toTable("bronze.events")
```

Handling Bad Records

Configure schema validation, quarantine invalid records, monitor the rescued data column, and create separate pipelines for data correction instead of stopping production ingestion.

Performance Optimization

Use File Notification mode whenever possible, optimize checkpoint storage, avoid tiny files, enable Photon, partition Delta tables appropriately, and periodically run OPTIMIZE and VACUUM.

Production Design Pattern

A production pipeline ingests files through Auto Loader into the Bronze layer, validates records in Silver, captures invalid data separately, and publishes curated Gold tables for reporting and machine learning.

Advanced Interview Questions

1. What is the purpose of the `_rescued_data` column?
2. When should you use Schema Hints?
3. How do you handle malformed records?
4. How does Auto Loader scale to millions of files?
5. What are the best practices for production Auto Loader pipelines?

Databricks Master Interview Handbook (2026)

Chapter 7 – Change Data Feed (CDF) (Part 1)

What is Change Data Feed (CDF)?

Change Data Feed (CDF) is a Delta Lake feature that captures row-level changes such as inserts, updates, and deletes. Instead of reading the entire table, downstream processes can read only the changed data, improving performance and reducing processing time.

How CDF Works

When CDF is enabled on a Delta table, every committed transaction records change information. Consumers can query only the modified rows between specific table versions or timestamps.

Enable CDF

```
CREATE TABLE silver.orders (  
  order_id INT,  
  customer_id INT,  
  amount DECIMAL(10,2)  
)  
USING DELTA  
TBLPROPERTIES (  
  delta.enableChangeDataFeed = true  
);
```

Read Changes Using SQL

```
SELECT *  
FROM table_changes('silver.orders', 5, 10);
```

PySpark Example

```
changes_df = (spark.read  
  .format("delta")  
  .option("readChangeFeed", "true")  
  .option("startingVersion", 5)  
  .table("silver.orders"))  
  
display(changes_df)
```

Real-World Scenario

A banking application processes millions of transactions daily. Instead of reprocessing the complete transactions table, downstream pipelines use CDF to process only newly inserted, updated, or deleted records.

Interview Questions

1. What is Change Data Feed?
2. Why should we use CDF?
3. How do you enable CDF?
4. How do you read changes using SQL and PySpark?
5. What are the advantages of CDF over full-table processing?

Databricks Master Interview Handbook (2026)

Chapter 7 – Change Data Feed (CDF) (Part 2)

CDF Metadata Columns

CDF adds metadata such as `_change_type`, `_commit_version` and `_commit_timestamp`. These fields help identify what changed and when the transaction was committed.

Change Types

```
insert
update_preimage
update_postimage
delete
```

Incremental ETL with CDF

Instead of reading an entire Delta table every run, ETL pipelines read only the changes since the last processed version. This significantly reduces compute time and cost.

PySpark Example

```
changes = (spark.read
    .format("delta")
    .option("readChangeFeed", "true")
    .option("startingVersion", 25)
    .table("silver.orders"))

changes.filter("_change_type = 'insert']").show()
```

Using CDF for SCD Type 2

CDF simplifies SCD Type 2 implementations by identifying updated records. Existing records can be expired and new versions inserted without scanning the full source table.

Performance Best Practices

Track the last processed version, process changes in small batches, combine CDF with MERGE, optimize Delta tables regularly, and archive old data when appropriate.

Production Architecture

Source systems write to Bronze, transformations populate Silver, CDF captures incremental changes, and Gold reporting tables are refreshed using only the changed records.

Advanced Interview Questions

1. What metadata columns are available in CDF?
2. Explain insert, update_preimage, update_postimage and delete.
3. How do you build incremental ETL using CDF?
4. How is CDF used in SCD Type 2?
5. What are the production best practices for CDF?

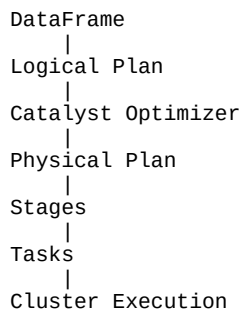
Databricks Master Interview Handbook (2026)

Chapter 8 – Performance Optimization (Part 1)

Spark Execution Plan

Every Spark job is converted into a logical plan, optimized by the Catalyst Optimizer, transformed into a physical plan, and then executed as stages and tasks across the cluster.

Execution Flow



Catalyst Optimizer

Catalyst automatically performs predicate pushdown, constant folding, projection pruning, join reordering, and other optimizations to generate an efficient execution plan.

Adaptive Query Execution (AQE)

AQE optimizes queries during runtime by dynamically changing join strategies, coalescing shuffle partitions, and handling skewed data based on actual execution statistics.

Partitioning

Choose partition columns with balanced cardinality and frequent filtering. Avoid too many tiny partitions or very large partitions that cause data skew.

Caching vs Persistence

Caching stores data in memory for faster reuse. Persistence supports additional storage levels such as memory-and-disk when datasets do not fit completely in memory.

PySpark Example

```
df = spark.table("silver.sales")
df.cache()
result = df.groupBy("country").sum("amount")
result.show()
```

Interview Questions

1. Explain the Spark execution plan.
2. What does Catalyst Optimizer do?

3. What is Adaptive Query Execution?
4. How do you choose partition columns?
5. Cache vs Persist?

Databricks Master Interview Handbook (2026)

Chapter 8 – Performance Optimization (Part 2)

Broadcast Join

Broadcast Join sends the smaller table to every executor, avoiding expensive shuffle operations. It is ideal when one table is small enough to fit in executor memory.

PySpark Example – Broadcast Join

```
from pyspark.sql.functions import broadcast

result = fact_df.join(
    broadcast(dim_df),
    "customer_id",
    "inner"
)
```

Shuffle Operations

Shuffle redistributes data across executors during joins, aggregations, DISTINCT, GROUP BY, and ORDER BY. Reducing unnecessary shuffles significantly improves performance.

Handling Data Skew

Identify skewed keys using execution metrics. Use salting, repartitioning, AQE skew handling, or broadcast joins to distribute data more evenly.

Repartition vs Coalesce

Repartition performs a full shuffle and can increase or decrease partitions. Coalesce primarily reduces partitions while minimizing shuffle, making it suitable before writing output files.

OPTIMIZE, ZORDER and Photon

OPTIMIZE compacts small files into larger ones. ZORDER colocates related data for faster filtering. Photon is Databricks' vectorized execution engine that accelerates SQL and Delta workloads.

Production Performance Tuning

Monitor Spark UI, choose appropriate cluster sizes, tune shuffle partitions, cache reusable datasets, optimize Delta tables regularly, and review query execution plans.

Advanced Interview Questions

1. When should you use a Broadcast Join?
2. What causes shuffle operations?
3. How do you handle data skew?
4. Repartition vs Coalesce?
5. Explain OPTIMIZE, ZORDER and Photon.

Databricks Master Interview Handbook (2026)

Chapter 9 – PySpark Deep Dive (Part 1)

RDD vs DataFrame vs Dataset

RDD is the low-level distributed collection with manual optimization. DataFrames provide a schema and are optimized by Catalyst. Datasets combine type safety with DataFrame optimizations (primarily in Scala/Java). In PySpark, DataFrames are the standard choice.

SparkSession

SparkSession is the entry point for all Spark applications. It is used to create DataFrames, execute SQL, configure Spark settings and access catalogs.

Transformations vs Actions

```
Transformations:
select(), filter(), join(), groupBy(), withColumn()

Actions:
show(), count(), collect(), write(), first()
```

Lazy Evaluation

Spark records transformations in a logical execution plan and executes them only when an action is called. This allows Spark to optimize the complete query before execution.

Narrow vs Wide Transformations

Narrow transformations process data within the same partition, while wide transformations require shuffling data across partitions. Wide transformations are generally more expensive.

PySpark Example

```
from pyspark.sql.functions import col

df = spark.table("sales")

result = (df.filter(col("amount") > 1000)
          .groupBy("country")
          .sum("amount"))

result.show()
```

Interview Questions

1. RDD vs DataFrame?
2. What is SparkSession?
3. Transformation vs Action?
4. Explain Lazy Evaluation.
5. Narrow vs Wide transformation?

Databricks Master Interview Handbook (2026)

Chapter 9 – PySpark Deep Dive (Part 2)

Joins in PySpark

PySpark supports inner, left, right, full, semi and anti joins. Choose broadcast joins for small lookup tables to reduce shuffle and improve performance.

Join Example

```
result = orders.join(
    customers,
    "customer_id",
    "left"
)
```

Window Functions

Window functions perform calculations across a set of rows while preserving row-level detail. Common functions include `row_number()`, `rank()`, `dense_rank()`, `lag()` and `lead()`.

Window Example

```
from pyspark.sql.window import Window
from pyspark.sql.functions import row_number

w = Window.partitionBy("dept").orderBy("salary")

df = df.withColumn("rn", row_number().over(w))
```

Built-in Functions vs UDF

Built-in Spark SQL functions are optimized by Catalyst and Photon. Use UDFs only when built-in functions cannot solve the requirement because UDFs generally reduce optimization opportunities.

Useful APIs

Frequently used APIs include `explode()`, `pivot()`, `unionByName()`, `withColumn()`, `when()`, `coalesce()`, `lit()`, `regexp_replace()` and `concat_ws()`.

Caching Example

```
sales = spark.table("silver.sales")
sales.cache()
sales.count()
```

Real-World Scenario

An Azure Databricks pipeline joins orders with customer and product dimensions, uses window functions to identify the latest records, applies built-in functions for transformations, and caches reusable datasets before writing Gold tables.

Advanced Interview Questions

1. Explain all join types in PySpark.
2. When do you use window functions?

3. UDF vs built-in functions?
4. What does `unionByName()` do?
5. When should you use `cache()` or `persist()`?

Databricks Master Interview Handbook (2026)

Complete Azure Data Engineering Interview Guide

Compiled from the uploaded handbook chapters.

Databricks Master Interview Handbook (2026)

Chapter 10 – SQL for Data Engineering (Part 1)

SQL Execution Order

The logical execution order is: FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT. Understanding this helps explain query behavior during interviews.

Execution Flow

```
FROM
JOIN
WHERE
GROUP BY
HAVING
SELECT
DISTINCT
ORDER BY
LIMIT
```

Joins

Common SQL joins are INNER, LEFT, RIGHT, FULL OUTER, CROSS, SELF, SEMI and ANTI joins. Choose the join type based on the business requirement and expected output.

Join Example

```
SELECT o.order_id,
       c.customer_name
FROM orders o
LEFT JOIN customers c
ON o.customer_id = c.customer_id;
```

Window Functions

Window functions calculate values across related rows without collapsing the result set. Frequently used functions include ROW_NUMBER(), RANK(), DENSE_RANK(), LAG() and LEAD().

ROW_NUMBER Example

```
SELECT *,
       ROW_NUMBER() OVER(
         PARTITION BY department
         ORDER BY salary DESC
       ) AS rn
FROM employees;
```

Common Interview Tips

Always write readable SQL, filter data as early as possible, avoid SELECT *, use proper indexing where applicable, and understand execution plans for performance tuning.

Interview Questions

1. Explain SQL execution order.
2. Explain different join types.
3. ROW_NUMBER vs RANK vs DENSE_RANK?

4. What are window functions?
5. How do you optimize SQL queries?

Databricks Master Interview Handbook (2026)

Chapter 10 – SQL for Data Engineering (Part 2)

Common Table Expressions (CTEs)

CTEs improve readability by breaking complex SQL into logical steps. They are temporary result sets available only during query execution.

CTE Example

```
WITH dept_sales AS (  
    SELECT department, SUM(amount) total_sales  
    FROM sales  
    GROUP BY department  
)  
SELECT *  
FROM dept_sales  
WHERE total_sales > 100000;
```

Views

Views store SQL logic as reusable virtual tables. Standard views execute the underlying query each time, while materialized views store precomputed results for faster reads where supported.

Subquery Example

```
SELECT employee_name  
FROM employees  
WHERE salary > (  
    SELECT AVG(salary)  
    FROM employees  
);
```

Set Operators

UNION removes duplicates, UNION ALL keeps duplicates, INTERSECT returns common rows, and EXCEPT (or MINUS in some databases) returns rows present in the first query but not the second.

Aggregation Example

```
SELECT department,  
COUNT(*) employees,  
SUM(salary) total_salary,  
AVG(salary) avg_salary  
FROM employees  
GROUP BY department;
```

Real-World SQL Scenario

A reporting pipeline uses CTEs to simplify business logic, joins fact and dimension tables, aggregates KPIs, and exposes reusable views for Power BI dashboards.

Advanced Interview Questions

1. What are CTEs and when do you use them?
2. View vs Materialized View?
3. Correlated vs non-correlated subquery?
4. UNION vs UNION ALL?

5. Explain GROUP BY and HAVING with examples.

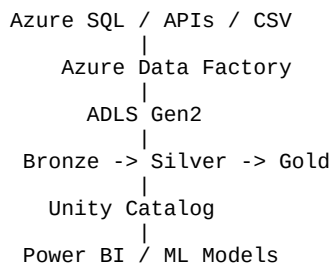
Databricks Master Interview Handbook (2026)

Chapter 11 – End-to-End Azure Databricks Project (Part 1)

Project Overview

This project ingests customer, product and sales data from Azure SQL, REST APIs and CSV files using Azure Data Factory into ADLS Gen2. Databricks transforms the data through Bronze, Silver and Gold layers and publishes curated tables for Power BI.

Architecture



ADF Copy Activity

```
{
  "source": {"type": "AzureSqlSource"},
  "sink": {"type": "DelimitedTextSink"}
}
```

PySpark - Bronze Load

```
from pyspark.sql.functions import current_timestamp

df = (spark.read
      .option("header", True)
      .csv("/mnt/raw/orders"))

(df.withColumn("ingestion_time", current_timestamp())
  .write.format("delta")
  .mode("append")
  .saveAsTable("bronze.orders"))
```

PySpark - Bronze to Silver

```
from pyspark.sql.functions import col

bronze = spark.table("bronze.orders")

silver = (bronze
         .filter(col("order_id").isNotNull())
         .dropDuplicates(["order_id"]))

silver.write.mode("overwrite").saveAsTable("silver.orders")
```

PySpark - Silver to Gold

```
from pyspark.sql.functions import sum

gold = (spark.table("silver.orders")
      .groupBy("country")
      .agg(sum("amount").alias("total_sales")))

gold.write.mode("overwrite").saveAsTable("gold.sales_summary")
```

Interview Questions

1. Explain this architecture end to end.
2. Why use Bronze, Silver and Gold?
3. Why Delta tables?
4. Where do you perform validations?
5. How is Power BI connected to Gold?

Databricks Master Interview Handbook (2026)

Chapter 11 – End-to-End Azure Databricks Project (Part 2)

Incremental Loading with Watermark

Read only records newer than the last processed timestamp to minimize processing time.

PySpark Code

```
from pyspark.sql.functions import col

last_watermark = "2026-07-30 00:00:00"

inc_df = (spark.read.table("bronze.orders")
          .filter(col("updated_at") > last_watermark))

inc_df.write.mode("append").saveAsTable("silver.orders_incremental")
```

SCD Type 2 using MERGE

Expire the existing active record and insert a new version whenever tracked columns change.

```
from delta.tables import DeltaTable

target = DeltaTable.forName(spark, "silver.customer_dim")

(target.alias("t")
 .merge(source_df.alias("s"), "t.customer_id=s.customer_id")
 .whenMatchedUpdate(
     condition="t.is_current=true AND t.hash<>s.hash",
     set={"is_current": "false", "end_date": "current_date()"}
 ).whenNotMatchedInsertAll()
 .execute())
```

Auto Loader + CDF

Use Auto Loader for continuous ingestion into Bronze and Change Data Feed to refresh downstream Silver/Gold incrementally instead of full reloads.

```
stream = (spark.readStream
 .format("cloudFiles")
 .option("cloudFiles.format", "json")
 .load("/mnt/raw/orders"))

(stream.writeStream
 .option("checkpointLocation", "/mnt/chk/orders")
 .toTable("bronze.orders"))
```

Error Handling

Validate mandatory fields, quarantine invalid records, log pipeline failures, and retry transient failures. Monitor with workflow alerts and event logs.

Production Best Practices

Use Unity Catalog, Photon, OPTIMIZE, ZORDER, partition large tables appropriately, parameterize notebooks, use ADF/Workflows orchestration, and implement CI/CD.

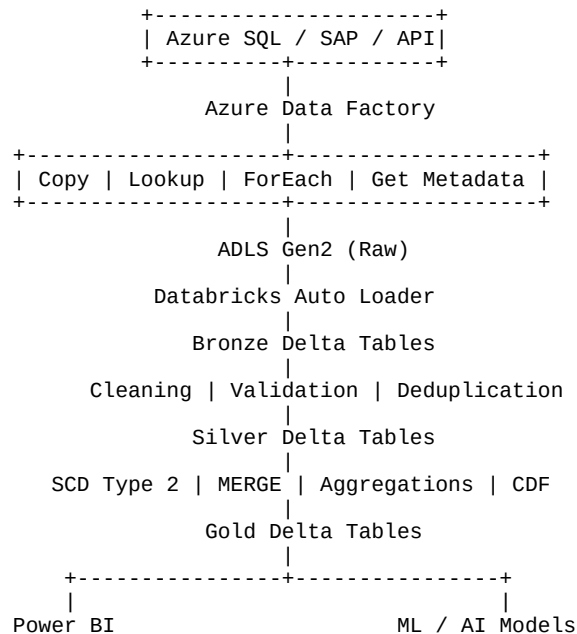
Interview Questions

1. How do you implement incremental loading?
2. Explain SCD Type 2 with MERGE.
3. How do Auto Loader and CDF work together?
4. How do you handle bad records?
5. How do you optimize this project for production?

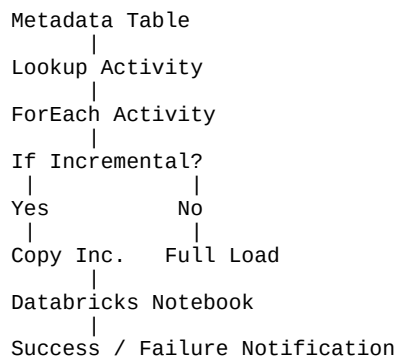
Databricks Master Interview Handbook (2026)

Chapter 12 – Azure Data Factory Real-Time Project (Part 1)

End-to-End Production Architecture



ADF Dynamic Pipeline Flow



Notebook Code Example

```
dbutils.widgets.text("table_name","")  
  
table = dbutils.widgets.get("table_name")  
  
df = spark.read.table(f"bronze.{table}")  
  
df.write.mode("overwrite").saveAsTable(f"silver.{table}")
```

Key Interview Topics

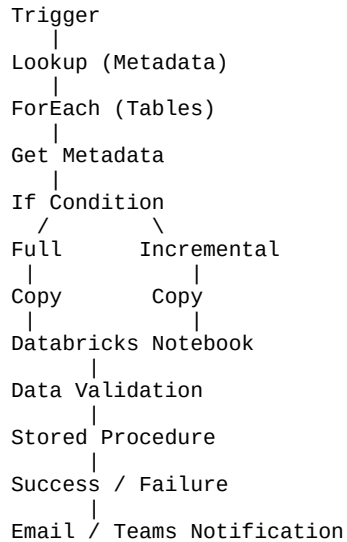
- Metadata-driven pipelines
- Dynamic parameterization
- Incremental loading
- Error handling and retries

- Monitoring with ADF and Databricks Workflows
- Unity Catalog security

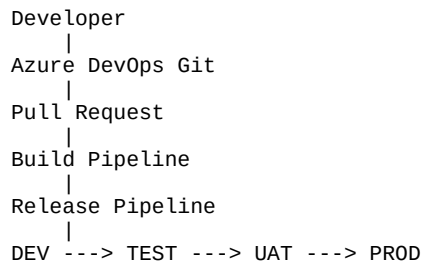
Databricks Master Interview Handbook (2026)

Chapter 12 – Azure Data Factory Real-Time Project (Part 2)

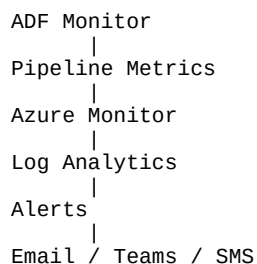
Complete ADF Activity Flow



CI/CD Deployment Flow



Production Monitoring Architecture



Databricks Notebook Example

```
dbutils.widgets.text("load_type","incremental")

load_type = dbutils.widgets.get("load_type")

if load_type == "incremental":
    print("Running Incremental Load")
else:
```

```
print("Running Full Load")
```

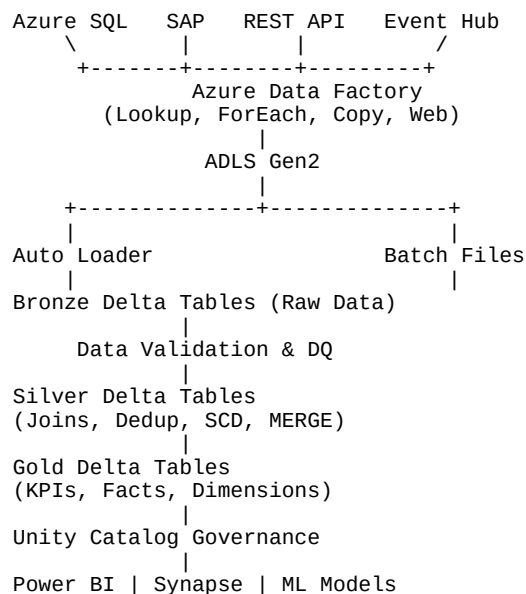
Production Best Practices

Use metadata-driven pipelines, parameterize datasets and linked services, implement retry policies, enable monitoring and alerting, use Unity Catalog for governance, and automate deployments using Azure DevOps CI/CD.

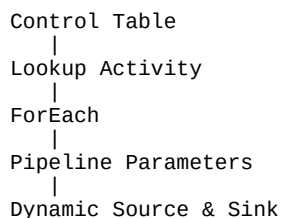
Databricks Master Interview Handbook (2026)

Chapter 13 – Complete Enterprise Azure Data Engineering Project (Part 1)

Enterprise Architecture



Metadata-Driven Framework



Production PySpark Example

```
from pyspark.sql.functions import current_timestamp

orders=(spark.read.table("bronze.orders")
        .dropDuplicates(["order_id"])
        .withColumn("processed_time",current_timestamp()))

orders.write.format("delta").mode("overwrite").saveAsTable("silver.orders")
```

Project Explanation

This enterprise solution ingests batch and streaming data from multiple source systems using Azure Data Factory and Event Hub. Data lands in ADLS Gen2 and is processed in Databricks through Bronze, Silver and Gold layers. Unity Catalog secures enterprise data, while curated Gold tables are consumed by Power BI, Synapse and machine learning workloads.

Interview Questions

1. Explain this architecture end to end.
2. Why use a metadata-driven framework?

3. Where is SCD Type 2 implemented?
4. How do batch and streaming coexist?
5. How is enterprise data secured?

Databricks Master Interview Handbook (2026)

Chapter 13 – Complete Enterprise Azure Data Engineering Project (Part 2)

Production Data Flow

```
Sources
|
ADF / Event Hub
|
Bronze (Raw)
|
Incremental Load (Watermark/CDF)
|
Silver (SCD Type 2 + MERGE)
|
Gold (KPIs & Reporting)
|
Power BI / ML / Synapse
```

Incremental Load (Watermark)

```
from pyspark.sql.functions import col

last_ts = "2026-07-31 00:00:00"

inc_df = (spark.table("bronze.orders")
          .filter(col("updated_at") > last_ts))

inc_df.write.mode("append").saveAsTable("silver.orders")
```

SCD Type 2 with MERGE

```
from delta.tables import DeltaTable

target = DeltaTable.forName(spark, "silver.customer_dim")

(target.alias("t")
 .merge(source_df.alias("s"),
        "t.customer_id=s.customer_id")
 .whenMatchedUpdate(
    condition="t.hash<>s.hash AND t.is_current=true",
    set={
        "is_current":"false",
        "end_date":"current_date()"
    })
 .whenNotMatchedInsertAll()
 .execute())
```

Monitoring & CI/CD

Use Azure Monitor, Log Analytics and Databricks Workflows for monitoring. Manage code in Azure DevOps or GitHub with pull requests, build pipelines, release pipelines and automated deployment to DEV, TEST, UAT and PROD.

Performance Checklist

- Enable Photon
- Use Auto Loader and CDF
- Partition large Delta tables
- Run OPTIMIZE and ZORDER
- Cache reused DataFrames
- Broadcast small dimension tables
- Monitor Spark UI regularly

Enterprise Interview Scenario

You are asked to design a pipeline that processes both batch and streaming data. Explain the ingestion layer, Medallion architecture, incremental processing, SCD Type 2, governance with Unity Catalog, monitoring, and CI/CD deployment. Highlight how you ensure scalability, reliability and cost optimization.

Azure Data Engineering Interview Sticky Notes

Quick Revision Sheet (5+ Years)

ADF	Lookup → ForEach → Copy → Notebook → Validation → Success/Failure
Incremental Load	Watermark, CDF, MERGE, Last Updated Timestamp
Bronze	Raw data, append only, audit trail
Silver	Clean, deduplicate, joins, business rules
Gold	KPIs, aggregates, reporting tables
Delta Lake	ACID, Time Travel, MERGE, CDF, OPTIMIZE, VACUUM
Unity Catalog	Central governance, RBAC, lineage
Auto Loader	cloudFiles, schema evolution, checkpoints
Spark	Lazy Evaluation, DAG, Stages, Tasks
Performance	Photon, Broadcast Join, AQE, ZORDER, Cache
PySpark	Prefer built-in functions over UDFs
SCD Type 2	Expire old row, insert new row, maintain history
SQL	FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY
Monitoring	ADF Monitor, Spark UI, Log Analytics
CI/CD	Git → PR → Build → Release → DEV → TEST → PROD

Interview Reminder: Always explain the architecture first, then ingestion, transformation, optimization, security, monitoring, and finally business outcome.

Azure Data Engineering Interview Sticky Notes

Quick Revision Sheet (Part 2)

One-Line Interview Answers

ADF	Used for orchestration and data movement.
Databricks	Performs distributed data processing using Spark.
Delta Lake	Adds ACID transactions to the data lake.
Auto Loader	Incrementally ingests new files efficiently.
CDF	Captures inserts, updates and deletes.
SCD Type 2	Maintains full historical data.
Photon	Vectorized engine for faster SQL execution.
AQE	Optimizes execution dynamically at runtime.
Broadcast Join	Broadcasts small tables to avoid shuffle.
Unity Catalog	Provides centralized governance and security.

PySpark Cheat Sheet

read(), write(), select(), filter(), withColumn(), groupBy(), agg(), join(), dropDuplicates(), unionByName(), explode(), when(), lit(), cache(), persist(), repartition(), coalesce().

SQL Cheat Sheet

Execution Order: FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT

Window Functions: ROW_NUMBER(), RANK(), DENSE_RANK(), LAG(), LEAD()

Set Operators: UNION, UNION ALL, INTERSECT, EXCEPT

Common Interview Mistakes

- Jumping into code before explaining architecture.
- Forgetting incremental loading strategy.
- Not discussing monitoring and error handling.
- Ignoring performance optimization.
- Not mentioning security and governance.

30-Minute Final Revision

Revise Medallion Architecture, ADF activities, Delta Lake, Unity Catalog, Auto Loader, CDF, Spark optimization, SCD Type 2, SQL window functions, and one end-to-end project explanation.

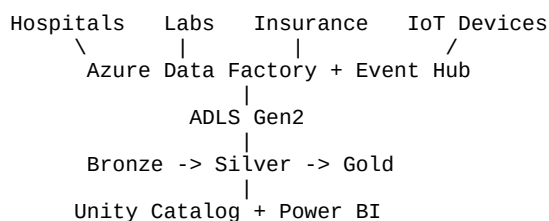
Databricks Master Interview Handbook (2026)

Chapter 15 – Real-Time Azure Data Engineering Case Studies (Part 3)

Case Study 3: Healthcare Analytics Platform

A healthcare organization ingests EHR records, laboratory results, insurance claims and IoT patient-monitoring data. ADF loads batch files while Event Hub streams device events. Databricks standardizes data in Bronze, Silver and Gold layers for clinical reporting and analytics.

Architecture



Disaster Recovery Strategy

Deploy resources across paired Azure regions, store Delta tables in geo-redundant storage, enable automated backups, use checkpoints for streaming jobs, and validate recovery using regular disaster recovery drills.

Cost Optimization

Use job clusters for scheduled workloads, enable autoscaling, terminate idle clusters automatically, optimize Delta tables, archive historical data, and schedule heavy processing during off-peak hours.

Interview Discussion

Explain data governance, patient-data security, HIPAA-style compliance concepts, monitoring, scalability, performance tuning, and how Medallion Architecture improves data quality before reporting.

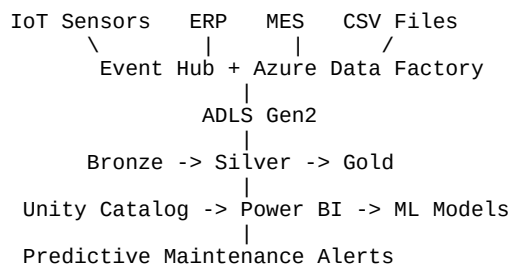
Databricks Master Interview Handbook (2026)

Chapter 15 – Real-Time Azure Data Engineering Case Studies (Part 4)

Case Study 4: Manufacturing IoT Platform

A global manufacturer collects telemetry from thousands of factory sensors. IoT events arrive continuously through Event Hub while ERP and production files are loaded in batches through Azure Data Factory. Databricks processes the data for predictive maintenance and production analytics.

Enterprise Architecture



Monitoring & Observability

Use Azure Monitor, Log Analytics, Databricks Workflows, Spark UI, and alerting for failed pipelines, slow jobs, cluster utilization, streaming lag, and data quality metrics.

Security & Governance

Implement Unity Catalog for centralized governance, role-based access control, column masking, audit logs, managed identities, Key Vault integration, and encryption at rest and in transit.

Senior Architect Scenario

Design a multi-region manufacturing platform that supports disaster recovery, scalable streaming ingestion, incremental processing, cost optimization, and secure access for engineering, operations, and analytics teams.

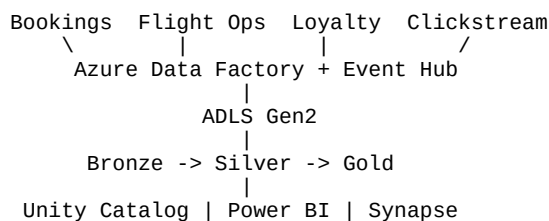
Databricks Master Interview Handbook (2026)

Chapter 15 – Real-Time Azure Data Engineering Case Studies (Part 5)

Case Study 5: Airline & Travel Analytics

An airline integrates booking systems, flight operations, loyalty programs and website clickstream data. Azure Data Factory ingests batch data while Event Hub captures real-time events. Databricks builds curated datasets for revenue management, operational dashboards and customer analytics.

Enterprise Architecture



Data Quality Framework

Validate schema, mandatory columns, null values, duplicates, referential integrity and business rules before promoting data from Bronze to Silver. Quarantine invalid records and publish quality metrics.

SLA & Incident Management

Define SLAs for pipeline completion, freshness and availability. Use Azure Monitor, Log Analytics and Databricks Workflows for alerting, maintain runbooks for incident response, and perform root-cause analysis after major failures.

Principal Data Engineer Interview Scenario

Explain how you would design a globally scalable platform supporting batch and streaming ingestion, secure governance with Unity Catalog, disaster recovery, CI/CD, monitoring, cost optimization and high availability while meeting strict business SLAs.

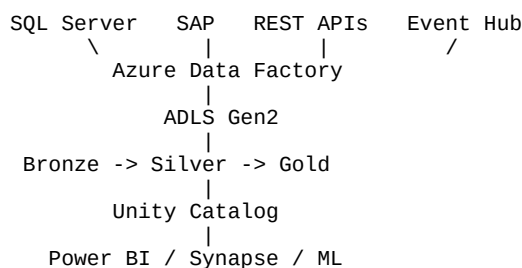
Databricks Master Interview Handbook (2026)

Chapter 16 – Azure Data Engineer Mock Interview Workbook (Part 1)

Mock Interview Scenario

Scenario: You are interviewing for a Senior Azure Data Engineer role with 5+ years of experience. The interviewer asks you to explain your latest end-to-end Azure Data Engineering project, covering architecture, ingestion, transformation, governance, monitoring, and optimization.

Architecture Whiteboard



Live Coding Challenge

```
from pyspark.sql.functions import col

orders = spark.table("bronze.orders")
result = orders.filter(col("status") == "Completed")
result.write.mode("overwrite").saveAsTable("silver.orders")
```

SQL Challenge

```
SELECT customer_id,
       SUM(amount) AS total_amount
FROM sales
GROUP BY customer_id
HAVING SUM(amount) > 10000;
```

Interviewer Follow-up Questions

• How do you handle incremental loading? • How do you recover from failed pipelines? • How do you optimize Spark jobs? • How do you secure data using Unity Catalog? • How do you monitor production pipelines?

Databricks Master Interview Handbook (2026)

Chapter 16 – Azure Data Engineer Mock Interview Workbook (Part 2)

Technical Manager Mock Interview

Question: Design an Azure Data Engineering platform processing 2 TB of data daily.

Expected Discussion: Explain source systems, Azure Data Factory orchestration, ADLS Gen2, Databricks Medallion Architecture, Delta Lake, Unity Catalog, monitoring, CI/CD, security, and cost optimization.

Spark Performance Troubleshooting

Problem:

A job that normally finishes in 40 minutes now takes 3 hours.

Investigation Steps:

1. Review Spark UI stages and tasks.
2. Check data skew and shuffle size.
3. Inspect cluster autoscaling.
4. Verify partition count.
5. Run OPTIMIZE and ZORDER if required.
6. Review recent schema or volume changes.

System Design Exercise

```
Sources
|
Azure Data Factory
|
ADLS Gen2
|
Bronze -> Silver -> Gold
|
Unity Catalog
|
Power BI / Synapse / ML
```

Production Incident Exercise

Scenario: A pipeline fails after loading half the records. Explain how idempotent design, checkpoints, watermark-based incremental loading, logging, retries, and validation prevent duplicates during reruns.

HR Mock Questions

- Describe a challenging production issue you solved.
- Why are you looking for a new role?
- How do you prioritize multiple production incidents?
- Describe a time you improved pipeline performance.
- How do you collaborate with QA, analysts, and business teams?

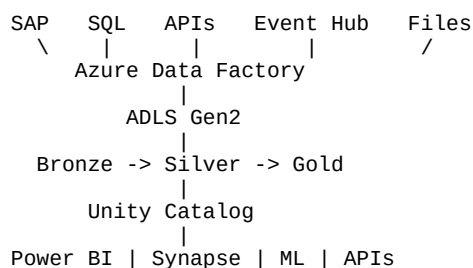
Databricks Master Interview Handbook (2026)

Chapter 16 – Azure Data Engineer Mock Interview Workbook (Part 3)

Director / Solution Architect Interview Simulation

Scenario: Design an enterprise Azure Data Platform that ingests 5+ TB of data per day from SAP, SQL Server, REST APIs, Kafka/Event Hub, and flat files. Discuss scalability, governance, security, disaster recovery, monitoring, CI/CD, and cost optimization.

Enterprise Architecture



Advanced PySpark Coding Exercise

```
from pyspark.sql.functions import broadcast

sales = spark.table("silver.sales")
products = broadcast(spark.table("gold.product_dim"))

result = (sales.join(products, "product_id", "left")
          .filter("amount > 1000"))

result.write.mode("overwrite").saveAsTable("gold.high_value_sales")
```

Advanced SQL Challenge

```
SELECT customer_id,
       SUM(amount) AS total_sales,
       DENSE_RANK() OVER(ORDER BY SUM(amount) DESC) AS sales_rank
FROM sales
GROUP BY customer_id;
```

Mock Interview Evaluation Scorecard

Evaluate yourself on: • Architecture explanation • Azure Data Factory knowledge • Databricks & Delta Lake • PySpark coding • SQL problem solving • Performance optimization • Communication and project explanation Score each area from 1 to 5 and identify topics that need revision before the interview.

Databricks Master Interview Handbook (2026)

Chapter 16 – Azure Data Engineer Mock Interview Workbook (Part 4)

Complete Mock Interview Flow

```
graph TD; HR[HR Round] --> Tech[Technical Round]; Tech --> SQL[Live SQL Coding]; SQL --> PySpark[PySpark Coding]; PySpark --> ADF[ADF & Databricks Design]; ADF --> Manager[Manager Round]; Manager --> SA[Solution Architect Round]; SA --> Final[Final Discussion];
```

Whiteboard Exercise

Design an Azure Data Platform that ingests batch and streaming data, implements Bronze, Silver, and Gold layers, uses Unity Catalog for governance, supports CI/CD, monitoring, disaster recovery, and cost optimization. Explain every design decision.

Advanced Troubleshooting Scenarios

1. Pipeline succeeds but target has duplicate records.
2. Spark job fails with executor out-of-memory errors.
3. Streaming job has increasing processing lag.
4. ADF pipeline times out while copying large files.
5. Gold layer metrics do not match source totals.

Rapid-Fire Questions

- Difference between repartition() and coalesce()?
- What is Delta Time Travel?
- When do you use broadcast joins?
- How do you implement watermark-based incremental loading?
- What is the purpose of OPTIMIZE and ZORDER?

Final Interview Readiness Checklist

- ✓ Explain your project confidently.
- ✓ Practice SQL and PySpark coding.
- ✓ Review Azure Data Factory activities.
- ✓ Understand Databricks optimization techniques.
- ✓ Revise Delta Lake, Unity Catalog, Auto Loader, and Change Data Feed.
- ✓ Be ready with real production examples and troubleshooting stories.

Databricks Master Interview Handbook (2026)

Chapter 16 – Azure Data Engineer Mock Interview Workbook (Part 5)

Complete Interview Simulation

Walk through your project in this order: business requirement, source systems, Azure Data Factory orchestration, ADLS Gen2, Medallion Architecture, Databricks transformations, Delta Lake, Unity Catalog, monitoring, CI/CD, performance tuning, security, and business outcome.

Evaluation Rubric

Area	Score (1-5)
Project Explanation	
Azure Data Factory	
Databricks	
PySpark	
SQL	
Performance Tuning	
Troubleshooting	
Communication	

Common Mistakes to Avoid

- Jumping into technical details before explaining the business requirement.
- Giving memorized answers without project examples.
- Ignoring monitoring, security, governance, and cost optimization.
- Not explaining why a particular Azure service was selected.

7-Day Revision Plan

Day 1: Azure Data Factory

Day 2: Databricks & Delta Lake

Day 3: PySpark Coding

Day 4: SQL Practice

Day 5: End-to-End Project

Day 6: Mock Interviews

Day 7: Final Revision & HR

Ultimate Cheat Sheet

Remember: Explain architecture first, justify design decisions, discuss optimization, monitoring, governance, and conclude with measurable business impact. Support every answer with a real production example.

Databricks Master Interview Handbook (2026)

Chapter 17 – Advanced PySpark & SQL Coding Workbook (Part 1)

Coding Scenario 1: Remove Duplicate Records

```
from pyspark.sql.window import Window
from pyspark.sql.functions import row_number

w = Window.partitionBy("customer_id").orderBy("updated_at")
latest = (df.withColumn("rn", row_number().over(w))
          .filter("rn = 1")
          .drop("rn"))
```

Coding Scenario 2: Incremental Load

```
from pyspark.sql.functions import col

watermark = "2026-07-31 00:00:00"

incremental_df = source_df.filter(col("last_updated") > watermark)
```

SQL Challenge

```
SELECT customer_id,
       SUM(amount) AS total_sales,
       ROW_NUMBER() OVER(
         ORDER BY SUM(amount) DESC
       ) AS rn
FROM sales
GROUP BY customer_id;
```

Interview Discussion

Explain why window functions are used for deduplication, how watermark-based incremental loading reduces processing time, and how these techniques improve scalability in production Azure Databricks pipelines.

Databricks Master Interview Handbook (2026)

Chapter 17 – Advanced PySpark & SQL Coding Workbook (Part 2)

Scenario 1: SCD Type 2 using Delta MERGE

```
from delta.tables import DeltaTable

target = DeltaTable.forName(spark, "silver.customer_dim")

(target.alias("t")
 .merge(source_df.alias("s"),
        "t.customer_id = s.customer_id")
 .whenMatchedUpdate(
    condition="t.hash <> s.hash AND t.is_current = true",
    set={
        "is_current": "false",
        "end_date": "current_date()"
    })
 .whenNotMatchedInsertAll()
 .execute())
```

Scenario 2: Complex Join & Aggregation

```
from pyspark.sql.functions import sum, broadcast

orders = spark.table("silver.orders")
customers = broadcast(spark.table("gold.customer_dim"))

result = (orders.join(customers, "customer_id")
 .groupBy("country")
 .agg(sum("amount").alias("total_sales")))
```

Advanced SQL Challenge

```
WITH monthly_sales AS (
SELECT customer_id,
       DATE_TRUNC('month', order_date) AS month,
       SUM(amount) AS sales
FROM orders
GROUP BY customer_id, DATE_TRUNC('month', order_date)
)
SELECT *
FROM monthly_sales
WHERE sales > 5000;
```

Performance Optimization Tips

Use broadcast joins for small dimensions, repartition before large shuffles, avoid unnecessary `cache()`, enable Photon, optimize Delta tables with `OPTIMIZE` and `ZORDER`, and use Adaptive Query Execution (AQE) where available.

Interview Discussion

Be ready to explain why MERGE is preferred for SCD Type 2, how broadcast joins reduce shuffle, and how Delta Lake features improve reliability and performance in enterprise data pipelines.

Databricks Master Interview Handbook (2026)

Chapter 18 – Azure Data Engineering Production Playbook (Part 1)

Production Support Workflow

```
Alert Trigger
|
Azure Monitor
|
Incident Created
|
Initial Triage
|
Root Cause Analysis
|
Fix / Validation
|
Business Sign-off
|
Post Incident Review
```

Severity Matrix

- P1:** Complete production outage.
- P2:** Critical pipeline delayed.
- P3:** Partial functionality impacted.
- P4:** Minor issue or enhancement request.

Root Cause Analysis Checklist

Verify pipeline status, cluster health, Spark UI, source availability, schema changes, permissions, storage availability, checkpoint folders, Delta logs, and recent deployments.

Monitoring Best Practices

Use Azure Monitor, Log Analytics, Databricks Workflows, alert rules, dashboards, pipeline metrics, cluster metrics, and automated notifications through Teams or email.

On-call Engineer Checklist

Acknowledge alerts quickly, communicate impact, collect logs, restore service safely, avoid duplicate processing, validate data quality, document the RCA, and identify preventive actions.

Databricks Master Interview Handbook (2026)

Chapter 18 – Azure Data Engineering Production Playbook (Part 2)

Production Incident: ADF Pipeline Failure

A scheduled pipeline fails because the source database is unavailable. Verify connectivity, review activity logs, retry the failed activity, validate the source after recovery, and rerun only the failed load to avoid duplicate processing.

Databricks Cluster Troubleshooting

Symptoms:

- Job stuck in Pending
- Executor OutOfMemory
- Long shuffle stages

Actions:

1. Review Spark UI
2. Check cluster autoscaling
3. Inspect driver/executor logs
4. Repartition skewed data
5. Run OPTIMIZE if needed

Streaming Recovery

Restart the streaming job from the existing checkpoint, verify Event Hub offsets, confirm schema compatibility, and validate processed records before releasing downstream reports.

SLA / SLO Dashboard

```
Pipeline Success Rate
|
Average Runtime
|
Data Freshness
|
Failed Jobs
|
Alert Response Time
```

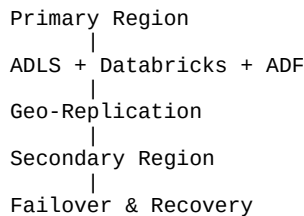
Production Support Interview Scenario

Explain how you would manage a P1 production incident, communicate with stakeholders, restore service safely, perform root cause analysis, document corrective actions, and implement preventive measures.

Databricks Master Interview Handbook (2026)

Chapter 18 – Azure Data Engineering Production Playbook (Part 3)

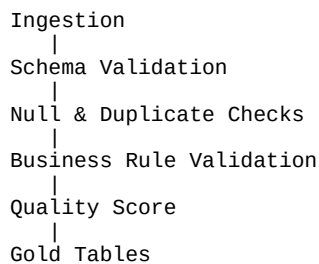
Multi-Region Disaster Recovery



Recovery Strategy

Use geo-redundant storage, infrastructure-as-code, automated backups, streaming checkpoints, Delta transaction logs, and documented failover procedures. Validate applications before switching users to the secondary region.

Data Quality Monitoring Framework



Production Cost Optimization

Use job clusters for scheduled workloads, enable autoscaling, terminate idle clusters, optimize Delta tables, archive cold data, right-size compute, and monitor storage and compute costs regularly.

Security Incident Response

Immediately isolate affected resources, rotate secrets through Azure Key Vault, review Unity Catalog audit logs, investigate access history, notify stakeholders, restore service, and document preventive actions.

Senior Production Support Case Study

A business-critical pipeline misses its SLA because of a sudden data volume increase. Explain how you would stabilize the platform, communicate status updates, optimize processing, validate data accuracy, and perform a complete root cause analysis.

Databricks Master Interview Handbook (2026)

Chapter 18 – Azure Data Engineering Production Playbook (Part 4)

Enterprise Monitoring Dashboard

```
Azure Monitor
|
Log Analytics
|
ADF Metrics | Databricks Jobs | Spark UI
|
Alerts -> Teams / Email / Pager
```

Capacity Planning

Forecast data growth, review cluster utilization, monitor storage trends, enable autoscaling, benchmark workloads quarterly, and tune partitioning before peak business periods.

Backup & Retention

Define retention for Bronze, Silver and Gold tables, protect Delta transaction logs, back up configuration metadata, and periodically test restoration procedures.

Compliance & Audit

Use Unity Catalog audit logs, RBAC, managed identities, Key Vault, encryption, and lineage reports to satisfy governance and compliance requirements.

Release & Change Management

```
Developer
|
Git Pull Request
|
CI Pipeline
|
DEV -> TEST -> UAT -> PROD
|
Post-Deployment Validation
```

Enterprise Support Interview Scenario

Describe how you would deploy a production change with zero data loss, monitor post-release health, execute rollback if required, and communicate status to business stakeholders.

Databricks Master Interview Handbook (2026)

Chapter 18 – Azure Data Engineering Production Playbook (Part 5)

Complete Production Operations Runbook

```
Receive Alert
|
Validate Incident
|
Assess Business Impact
|
Assign Severity (P1-P4)
|
Restore Service
|
Validate Data
|
Business Confirmation
|
RCA & Preventive Actions
```

P1 Major Incident Checklist

Engage the incident response team immediately, notify stakeholders, isolate the issue, restore critical services, verify data integrity, monitor stability, and conduct a detailed post-incident review with preventive action items.

Production Readiness Checklist

Confirm monitoring, alerting, backups, disaster recovery, security controls, access management, data validation, documentation, deployment approvals, rollback plans, and SLA compliance before release.

Operational Excellence Best Practices

Automate repetitive tasks, maintain Infrastructure as Code, optimize Databricks jobs regularly, review costs, perform health checks, conduct periodic DR drills, and continuously improve monitoring.

Enterprise KPIs

- Pipeline Success Rate
- SLA Compliance
- Mean Time to Detect (MTTD)
- Mean Time to Recover (MTTR)
- Data Freshness
- Cost per Pipeline Run
- Failed Job Percentage

Final Production Support Cheat Sheet

Always prioritize service restoration, communicate clearly, validate business data after every fix, document the RCA, implement preventive measures, and use automation to reduce future operational risk.

Databricks Master Interview Handbook (2026)

Chapter 19 – Azure Data Engineer Interview Master Questions (Part 1)

1. Explain the Azure Data Factory architecture.

ADF consists of pipelines, activities, datasets, linked services, integration runtimes, and triggers. Pipelines orchestrate data movement and transformation between different systems.

2. What is the difference between Azure Blob Storage and ADLS Gen2?

ADLS Gen2 builds on Blob Storage and adds a hierarchical namespace, making it more suitable for analytics workloads and big data processing.

3. Explain full load vs incremental load.

A full load copies the entire dataset every run, while an incremental load copies only new or changed records using timestamps, watermark columns, or CDC.

4. What are the Bronze, Silver, and Gold layers?

Bronze stores raw data, Silver stores cleansed and transformed data, and Gold contains business-ready curated datasets for reporting.

5. How do you optimize a Databricks job?

Use partitioning, broadcast joins, Adaptive Query Execution, Photon, caching when appropriate, Delta OPTIMIZE, ZORDER, and autoscaling job clusters.

Interview Tip

Answer with real project examples. Explain what you implemented, why you chose that solution, and how you handled failures, monitoring, and performance optimization.

Databricks Master Interview Handbook (2026)

Chapter 19 – Azure Data Engineer Interview Master Questions (Part 2)

1. How do you implement incremental loading in Azure Data Factory?

Use a watermark column such as last_updated_date or Change Data Capture (CDC). Store the last successful watermark and use it in the next pipeline run to load only new or modified records.

2. How do you handle pipeline failures?

Configure retry policies, use Try-Catch patterns with Execute Pipeline activities, log failures, send alerts through Azure Monitor, and restart only the failed step after validation.

3. Explain Delta Lake ACID transactions.

Delta Lake provides atomicity, consistency, isolation, and durability using a transaction log. This enables reliable updates, deletes, merges, and concurrent reads and writes.

4. How would you optimize a PySpark job processing billions of records?

Reduce shuffles, use appropriate partitioning, broadcast small tables, filter early, avoid unnecessary caching, enable Adaptive Query Execution, and optimize Delta tables regularly.

5. Production Scenario

A daily pipeline suddenly takes three hours instead of thirty minutes. Explain how you would investigate Spark UI, cluster metrics, data skew, source changes, partitioning, and recent deployments before applying fixes.

Interview Tip

Support every answer with a real project example and mention monitoring, scalability, performance tuning, and business impact.

Databricks Master Interview Handbook (2026)

Chapter 19 – Azure Data Engineer Interview Master Questions (Part 3)

1. How do you handle data skew in PySpark?

Identify skewed keys using Spark UI, repartition data, use salting for highly skewed keys, and broadcast small lookup tables where appropriate to reduce shuffle time.

2. Explain `row_number()`, `rank()`, and `dense_rank()`.

`row_number()` assigns a unique number to every row. `rank()` leaves gaps after ties. `dense_rank()` does not leave gaps after ties.

3. How do you optimize Delta Lake tables?

Run OPTIMIZE to compact small files, use ZORDER on frequently filtered columns, VACUUM old files, and partition tables only when it benefits query performance.

4. Design an Azure Data Engineering architecture.

A common architecture uses Azure Data Factory for orchestration, ADLS Gen2 for storage, Databricks for transformations, Delta Lake for reliable storage, Synapse or Power BI for analytics, and Azure Monitor for operational visibility.

5. Managerial Interview Question

How do you manage multiple production issues? Prioritize incidents based on business impact, communicate regularly with stakeholders, delegate tasks, restore critical services first, and complete a root cause analysis after resolution.

Interview Tip

Keep answers concise, explain the business value of your solution, and support technical explanations with examples from your projects.

Databricks Master Interview Handbook (2026)

Chapter 19 – Azure Data Engineer Interview Master Questions (Part 4)

1. How do you optimize a complex SQL query?

Review the execution plan, create appropriate indexes where applicable, filter data as early as possible, avoid SELECT *, use joins efficiently, and partition large tables when supported.

2. ADF Case Study: Pipeline Timeout

If a Copy Activity times out, verify source performance, Integration Runtime health, network connectivity, parallel copy settings, and retry policies before rerunning the failed activity.

3. Databricks Troubleshooting

For slow jobs, inspect Spark UI, identify skewed partitions, review executor logs, optimize joins, compact small files using OPTIMIZE, and confirm adequate cluster resources.

4. End-to-End Project Discussion

Explain the complete flow from source systems to ADLS Gen2, Azure Data Factory orchestration, Databricks transformations, Delta Lake storage, Gold layer creation, and Power BI reporting.

5. Client Communication Question

When communicating with business stakeholders, explain technical issues in simple language, provide clear timelines, share progress updates regularly, and confirm successful resolution after validation.

Interview Tip

Structure every answer as: problem, solution, implementation, result, and lessons learned. This demonstrates both technical depth and communication skills.

Databricks Master Interview Handbook (2026)

Chapter 19 – Azure Data Engineer Interview Master Questions (Part 5)

Mock Interview: Tell me about yourself.

Start with your experience, summarize your Azure Data Engineering skills, explain your recent project, and finish with how your work delivered business value.

HR Question: Why should we hire you?

Highlight your Azure Data Factory, Databricks, PySpark, SQL, Delta Lake, and production support experience. Emphasize problem-solving, collaboration, and continuous learning.

Leadership Scenario

Describe a situation where you prioritized production incidents, coordinated with developers and business teams, and ensured timely resolution while maintaining clear communication.

Client-Facing Question

When discussing technical issues with clients, use simple language, explain the impact, present the recovery plan, and provide regular progress updates until closure.

Salary Negotiation Tip

Research the market, discuss your skills and achievements confidently, focus on the value you bring, and negotiate professionally with flexibility.

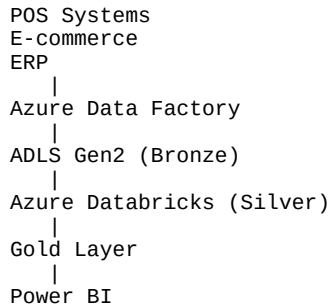
Ultimate Interview Cheat Sheet

Revise Azure Data Factory, Databricks, PySpark, SQL, Delta Lake, Spark optimization, Medallion architecture, CI/CD, monitoring, production support, and be ready with real project examples for every topic.

Databricks Master Interview Handbook (2026)

Chapter 20 – Azure Data Engineer Real-Time Case Studies (Part 1)

Case Study 1: Retail Data Platform



Challenge: Integrate sales, inventory, and customer data from multiple systems. Solution: Build a medallion architecture with incremental ingestion, Delta Lake, and automated quality checks.

Case Study 2: Banking Transaction Analytics

Use Event Hubs for streaming transactions, Databricks Structured Streaming for transformations, Delta Lake for storage, and Synapse/Power BI for fraud analytics dashboards.

Common Production Challenges

Schema drift, duplicate records, late-arriving data, skewed partitions, slow jobs, source outages, and storage growth. Address these using Auto Loader, Change Data Feed, partition optimization, monitoring, and retry strategies.

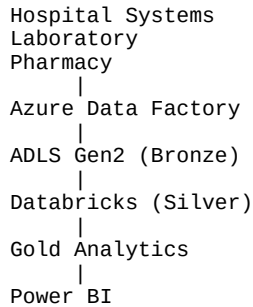
Interview Discussion Points

Explain business requirements, architecture decisions, technology selection, data flow, monitoring, security, scalability, and measurable business outcomes.

Databricks Master Interview Handbook (2026)

Chapter 20 – Azure Data Engineer Real-Time Case Studies (Part 2)

Case Study 3: Healthcare Data Platform



Challenge: Integrate patient, laboratory, and pharmacy data while maintaining data quality and security. Solution: Use Delta Lake, Unity Catalog, encryption, and automated validation rules.

Case Study 4: Insurance Claims Processing

Build pipelines to ingest policy, claims, and customer data. Implement incremental loading, SCD Type 2 for policy history, and Gold tables for claims reporting and fraud analysis.

Real-Time Data Ingestion Pattern

Capture streaming events using Event Hubs, process them with Databricks Structured Streaming, store data in Delta Lake, and expose curated datasets for analytics.

Data Quality & Governance

Perform schema validation, null checks, duplicate detection, business rule validation, and maintain data lineage using Unity Catalog.

Performance Optimization

Optimize Delta tables, use partitioning wisely, enable Photon and Adaptive Query Execution, and monitor cluster performance through Spark UI.

Scenario-Based Interview Question

Design a scalable Azure Data Engineering solution for processing millions of healthcare records daily while ensuring security, compliance, and low-latency reporting.

Databricks Master Interview Handbook (2026)

Chapter 20 – Azure Data Engineer Real-Time Case Studies (Part 3)

Case Study 5: E-Commerce Analytics Platform



Build an end-to-end analytics platform for orders, customers, products, and payments using incremental ingestion, Delta Lake, and Medallion architecture.

Case Study 6: Telecom Analytics

Ingest call detail records (CDRs), network events, and customer usage data. Process large volumes using Databricks, optimize with partitioning, and create Gold datasets for network and customer analytics.

IoT Streaming Architecture

Collect sensor events through Azure Event Hubs, process them with Structured Streaming, store them in Delta Lake, and expose near real-time dashboards for operational monitoring.

CI/CD for Azure Data Engineering

Use Git for version control, Azure DevOps pipelines for automated deployment, environment-specific configuration, unit testing, and controlled releases from DEV to TEST to PROD.

Cost Optimization Case Study

Reduce costs by using job clusters, autoscaling, Photon, Delta OPTIMIZE, storage lifecycle policies, and continuous monitoring of compute utilization.

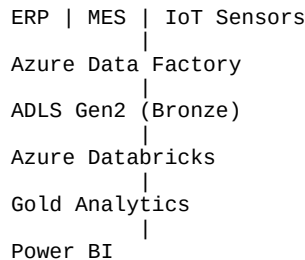
Production Troubleshooting Scenario

A nightly ETL misses its SLA after a sudden increase in data volume. Explain how you would identify the bottleneck, optimize processing, validate outputs, and prevent future occurrences.

Databricks Master Interview Handbook (2026)

Chapter 20 – Azure Data Engineer Real-Time Case Studies (Part 4)

Case Study 7: Manufacturing Analytics



Build a centralized manufacturing analytics platform to monitor production efficiency, machine utilization, quality metrics, and downtime using batch and streaming data.

Case Study 8: Supply Chain Data Platform

Integrate procurement, warehouse, inventory, shipment, and vendor data. Implement incremental ingestion, Delta Lake, and Gold dashboards for inventory optimization and delivery performance.

Finance & Accounting Analytics

Create pipelines for general ledger, accounts payable, accounts receivable, and budgeting data. Provide curated datasets for financial reporting and executive dashboards.

Enterprise Security Implementation

Secure the platform using Unity Catalog, Azure Key Vault, managed identities, RBAC, encryption at rest and in transit, audit logging, and data masking for sensitive information.

High Availability Architecture

Deploy services across multiple availability zones, enable geo-redundant storage, automate backups, and maintain documented disaster recovery procedures with periodic failover testing.

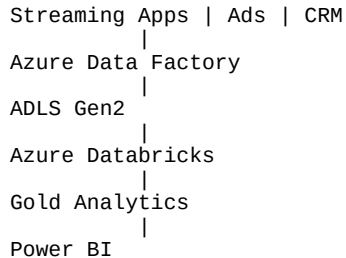
Advanced Interview Case Study

Design an enterprise-scale Azure Data Engineering platform handling both batch and streaming workloads for global operations while meeting strict security, scalability, and SLA requirements.

Databricks Master Interview Handbook (2026)

Chapter 20 – Azure Data Engineer Real-Time Case Studies (Part 5)

Case Study 9: Media & Entertainment Analytics



Build pipelines to analyze viewer engagement, subscriptions, advertising revenue, and content performance using Delta Lake and Medallion architecture.

Case Study 10: Airline & Travel Platform

Integrate reservation systems, flight operations, baggage tracking, and customer loyalty data. Create Gold datasets for operational dashboards, demand forecasting, and customer analytics.

Enterprise Reference Architecture

Use Azure Data Factory for orchestration, ADLS Gen2 for storage, Databricks for processing, Unity Catalog for governance, Delta Lake for reliable storage, and Power BI or Synapse for reporting.

Production Optimization Checklist

Monitor pipelines, optimize Delta tables, control cluster costs, validate data quality, automate deployments, enable alerting, and review security and access policies regularly.

Interview Case Study

Explain how you would design an enterprise-scale Azure Data Engineering platform capable of handling batch and streaming data, ensuring high availability, disaster recovery, governance, and cost optimization.

Final Real-World Cheat Sheet

Focus on architecture decisions, business requirements, scalability, monitoring, security, performance tuning, production support, and measurable business outcomes when answering case-study questions.